

CAHIER DES CHARGES TECHNIQUES

Projet myls

Cours : **Théorie des Systèmes d'Exploitation**

Classe : **INGE 3 SRT**

Année académique : **2025-2026**

Contents

Introduction.....	4
1. Contexte du projet	5
2. Problématique	5
3. Objectifs du projet.....	6
4. Livrables attendus.....	6
5. Besoins fonctionnels	7
6. Utilisateurs cibles	7
7. Syntaxe de la commande	8
8. Fonctionnement général de ls	8
9. Interface utilisateur et sortie	9
10. Exigences techniques	9
11. Contraintes d'implémentation	9
12. Architecture logicielle	10
13. Cycle de traitement	10
14. Gestion des processus	10
15. Arguments de ligne de commande	11
16. Appels POSIX	11
17. Système de fichiers	11
18. Permissions et protection	12
19. Gestion mémoire	12
20. Gestion des erreurs	13
21. Codes de retour	13
22. Tri des entrées	13
23. Affichage	14
24. Compilation et exécution	14
25. Tests de validation	14
26. Travail collaboratif	15
27. Répartition des modules par membre	16

28. Difficultés rencontrées	17
29. Solutions adoptées.....	18
30. Limites actuelles	19
31. Perspectives d'amélioration	20
Conclusion.....	21

Introduction

Le projet myls a été conçu pour reproduire, en langage C, une partie significative du comportement de la commande Linux ls. L'intérêt pédagogique du projet est double : comprendre l'usage d'un utilitaire système et mettre en évidence les mécanismes internes du système d'exploitation qui rendent ce type de commande possible.

Le document est rédigé comme un cahier des charges techniques afin de mettre l'accent sur la dimension système : création d'un processus, analyse des arguments, appel aux fonctions POSIX, lecture des répertoires, récupération des métadonnées, gestion des permissions, tri et affichage dans le terminal.

La commande ls est une référence classique sous Linux car elle montre que l'information visible dans le terminal provient directement du système de fichiers et des services exposés par le noyau. myls reprend cette logique dans une forme simplifiée, mais suffisamment complète pour illustrer les principes étudiés en théorie des systèmes d'exploitation.

1. Contexte du projet

Le projet est réalisé dans le cadre du cours de Théorie des Systèmes d'Exploitation. Il s'agit de répliquer une commande fondamentale de l'environnement Linux, à savoir ls, sous le nom myls, afin de relier la théorie à une implémentation concrète.

Le langage utilisé est le C, car il permet de manipuler directement les structures de bas niveau, la mémoire dynamique et les interfaces système. Cet aspect est essentiel pour comprendre comment les outils Unix et Linux sont construits.

Le projet donne également l'occasion d'interagir avec un système de fichiers réel. L'utilisateur demande au programme d'explorer des répertoires, et le programme interroge ensuite le système d'exploitation pour obtenir les entrées et leurs attributs.

2. Problématique

La question centrale est la suivante : comment reproduire en langage C le comportement essentiel de ls tout en interagissant correctement avec le système d'exploitation à travers les appels POSIX ?

Cette problématique ne concerne pas seulement l'affichage de noms de fichiers. Elle implique aussi la gestion du contexte d'exécution, le passage en mode noyau, l'accès aux métadonnées, les contrôles d'accès et la robustesse face aux erreurs.

Le projet montre ainsi qu'un utilitaire apparemment simple repose en réalité sur un ensemble de mécanismes système complexes, mais cohérents.

3. Objectifs du projet

Les objectifs sont de comprendre le fonctionnement interne de ls, de lire le contenu d'un répertoire Linux, de récupérer les métadonnées des fichiers et de gérer proprement les options de la ligne de commande.

Le projet vise également à trier et afficher les fichiers selon plusieurs critères, à produire des diagnostics lisibles en cas d'erreur et à développer un programme portable dans un environnement Linux standard.

Un autre objectif est de mettre en pratique les notions de processus, de mémoire, de permissions, de gestion des erreurs et de séparation stdout/stderr.

4. Livrables attendus

Les livrables du projet comprennent le code source C, l'exécutable myls, le Makefile, le rapport technique, la présentation orale, les tests de validation et le dépôt GitHub.

Cette organisation permet de couvrir l'ensemble du cycle de développement : écriture, compilation, validation, documentation et intégration collaborative.

Le rapport technique occupe ici un rôle central, car il doit justifier le choix de l'architecture, expliquer les mécanismes système utilisés et montrer la conformité du projet aux objectifs du cours.

5. Besoins fonctionnels

Le programme doit permettre à un utilisateur de lister le contenu d'un répertoire Linux avec une commande proche de ls. La syntaxe générale est : `./myls [OPTIONS] [CHEMIN...]`.

Le comportement par défaut consiste à afficher le répertoire courant lorsque aucun chemin n'est fourni. En présence de chemins multiples, chaque répertoire doit être traité proprement et affiché de manière compréhensible.

Les options de base incluent l'affichage des fichiers cachés, le format long, les tris différents, l'inversion de l'ordre et la récursivité lorsque celle-ci est supportée.

6. Utilisateurs cibles

Le programme s'adresse en priorité aux étudiants du cours, aux enseignants, aux développeurs débutants en systèmes et aux utilisateurs Linux souhaitant comprendre le fonctionnement interne de ls.

La simplicité de la commande myls doit permettre une prise en main rapide, tout en restant suffisamment riche pour montrer le lien entre le programme et le système d'exploitation.

Le public visé a donc besoin d'une solution claire, stable et techniquement expliquée.

7. Syntaxe de la commande

La syntaxe retenue est : `./mys [OPTIONS] [CHEMIN...]`. Cette forme imite la pratique courante des utilitaires Linux et permet une utilisation intuitive dans le terminal.

Exemples : `./mys`, `./mys -l`, `./mys -a`, `./mys -la`, `./mys -t`, `./mys -S`, `./mys /home/gunter`.

Le parsing doit distinguer les options des chemins, reconnaître les options combinées et conserver la logique de traitement dans un ordre cohérent.

8. Fonctionnement général de ls

Le fonctionnement logique d'un utilitaire de type `ls` peut être résumé ainsi : l'utilisateur saisit une commande, le shell lance le processus correspondant, le programme lit le système de fichiers, récupère les métadonnées, trie les résultats puis affiche le tout dans le terminal.

Ce déroulement met en évidence la chaîne classique `shell → processus → appels système → noyau → système de fichiers → retour de données → affichage`.

Cette chaîne est au cœur de la compréhension des systèmes d'exploitation, car elle montre que l'application ne fait qu'orchestrer des services fournis par le système.

9. Interface utilisateur et sortie

Le programme doit écrire les résultats normaux sur stdout afin de rester compatible avec les usages des tubes, des redirections et des scripts shell.

Les messages d'erreur doivent être envoyés vers stderr pour ne pas se mélanger à la sortie principale.

Cette séparation des flux est fondamentale dans toute programmation système, car elle permet de distinguer les données utiles des diagnostics.

10. Exigences techniques

Le projet doit être développé sous Linux en langage C avec GCC et un Makefile. GCC effectue les étapes classiques de compilation, d'assemblage et de liaison [web:3].

Le code doit être compilé avec des options de vérification comme -Wall et -Wextra afin de détecter rapidement les erreurs de syntaxe, les conversions implicites et les variables inutilisées.

Le projet doit rester autonome, sans dépendances externes lourdes, afin de conserver une structure simple et adaptée à un cours de systèmes.

11. Contraintes d'implémentation

Le programme doit être modulaire : chaque fonctionnalité importante doit être isolée dans un module dédié. Cette contrainte facilite la maintenance, la lecture du code et la collaboration entre plusieurs développeurs.

La mémoire doit être gérée manuellement. Les structures allouées pour stocker les entrées d'un répertoire doivent être libérées correctement après utilisation.

Le comportement du programme doit rester cohérent sur les chemins valides comme sur les erreurs d'accès ou d'allocation.

12. Architecture logicielle

L'architecture peut être découpée en `main.c`, `options.c/h`, `directory.c/h`, `entry.c/h`, `sort.c/h`, `display.c/h`, `utils.c/h` et `error.c/h`. Cette séparation correspond à une organisation classique en programmation système.

`main.c` reçoit `argc` et `argv`, initialise les options, lance le parsing, décide du traitement du répertoire courant ou des chemins fournis et coordonne la fin d'exécution.

Le module `options` gère l'analyse des drapeaux, `directory` lit les répertoires, `entry` récupère les métadonnées, `sort` classe les résultats, `display` formate l'affichage et `error` centralise les messages d'erreur.

13. Cycle de traitement

Le cycle d'exécution peut être décrit ainsi : lecture des arguments, initialisation des options, analyse des options, détermination des chemins, ouverture du répertoire, lecture des entrées, création des structures, tri, affichage, libération mémoire puis retour du code d'état.

Ce cycle est important parce qu'il relie la structure du programme aux mécanismes du système d'exploitation. Chaque étape correspond à une tâche précise qui doit être correctement ordonnancée.

Le programme ne doit pas seulement fonctionner ; il doit aussi montrer une démarche de traitement claire et systématique.

14. Gestion des processus

L'exécution de `./mys` déclenche la création d'un processus par le shell. Ce processus dispose de son propre espace d'adressage et s'exécute sous le contrôle du système d'exploitation.

Le noyau alloue les ressources nécessaires, planifie l'exécution du processus et lui permet d'accéder au système de fichiers à travers des mécanismes protégés.

Ce point relie directement le projet à la notion de processus étudiée en cours : un programme devient une instance vivante, planifiée et surveillée par le système.

15. Arguments de ligne de commande

Le programme exploite `argc` et `argv` pour interpréter les options et les chemins. Cela permet d'automatiser l'utilisation de la commande sans saisie interactive supplémentaire.

Par exemple, `./mys -la /home/gunter` produit typiquement `argv[0] = ./mys`, `argv[1] = -la` et `argv[2] = /home/gunter`.

Le parsing doit reconnaître les options combinées, les arguments invalides et les cas où aucun chemin n'est indiqué.

16. Appels POSIX

`mys` passe par des fonctions POSIX telles que `opendir()`, `readdir()`, `closedir()`, `lstat()`, `stat()`, `getpwuid()`, `getgrgid()` et `perror()`.

`opendir()` ouvre un flux de répertoire, `readdir()` lit successivement les entrées, `closedir()` ferme le flux, et `lstat()` récupère les métadonnées sans nécessairement suivre les liens symboliques.

Ce passage par les bibliothèques système démontre clairement que le programme travaille avec les services fournis par Linux et non avec le disque brut.

17. Système de fichiers

Le projet manipule les notions de fichiers ordinaires, répertoires, liens symboliques, permissions, inodes, tailles, dates, UID et GID. Ces éléments constituent le vocabulaire fondamental du système de fichiers Linux.

La structure `stat` permet d'accéder à des informations essentielles telles que `st_mode`, `st_size`, `st_mtime`, `st_uid`, `st_gid` et `st_nlink`.

Ces métadonnées sont indispensables pour produire un affichage détaillé, fidèle à ce que propose `ls`.

18. Permissions et protection

Linux applique un modèle de permissions fondé sur read, write et execute pour trois catégories : user, group et others. Ce mécanisme protège les données et contrôle les actions autorisées.

Si l'utilisateur ne dispose pas des droits nécessaires, `opendir()` ou `lstat()` peut échouer. Le programme doit alors réagir proprement et signaler le problème.

Cette exigence rappelle que les erreurs d'accès ne sont pas seulement des cas techniques ; elles résultent du modèle de sécurité du système d'exploitation.

19. Gestion mémoire

Le nombre d'entrées d'un répertoire n'étant pas connu à l'avance, le programme doit utiliser un tableau dynamique. Cela implique une allocation initiale, une croissance progressive et une libération finale.

`malloc()` sert à réserver l'espace, `realloc()` à l'agrandir, `strdup()` à copier les noms de fichiers et `free()` à libérer les ressources.

Une bonne gestion mémoire est essentielle dans un projet système, car elle évite les fuites et les comportements indéfinis.

20. Gestion des erreurs

Les erreurs courantes comprennent l'option inconnue, le chemin inexistant, la permission refusée, l'erreur d'ouverture, l'erreur de lecture, l'erreur mémoire et l'erreur de récupération des métadonnées.

`perror()` exploite `errno` pour produire un diagnostic lisible sur `stderr`.

Le programme doit aussi définir des codes de retour cohérents afin de signaler le succès ou l'échec de l'exécution.

21. Codes de retour

Un code de retour à 0 indique un succès. Un code à 1 peut représenter une erreur mineure, tandis qu'un code à 2 peut signaler une mauvaise utilisation ou une erreur plus grave.

Les codes doivent rester simples mais suffisamment explicites pour être utiles dans un contexte Unix.

Ils constituent aussi une interface entre le programme et le shell.

22. Tri des entrées

Le tri peut être réalisé selon plusieurs critères : nom, date, taille ou extension. L'option `-r` inverse ensuite l'ordre obtenu.

Le choix du critère doit dépendre de l'option la plus prioritaire. Le programme doit conserver une logique stable, même lorsque plusieurs options sont combinées.

Le tri est une fonction d'interface importante, car il influence directement la lisibilité de l'affichage.

23. Affichage

En mode simple, le programme affiche uniquement le nom des fichiers. En mode long, il affiche un ensemble d'informations structurées : permissions, nombre de liens, propriétaire, groupe, taille, date et nom.

Le format long doit s'approcher autant que possible de la présentation de `ls -l`, tout en restant compatible avec la simplicité du projet pédagogique.

L'affichage constitue la partie visible du travail, mais il repose en réalité sur tout le traitement système effectué en amont.

24. Compilation et exécution

La compilation se fait avec `make` ou `make re`. Le nettoyage se fait avec `make clean` et `make fclean`. Ce cycle reflète l'organisation classique d'un projet C sous Linux.

Le programme s'exécute avec `./mys` suivi des options choisies. Il est utile de comparer les résultats à `ls` sur les mêmes répertoires afin de valider la cohérence.

Les commandes de compilation et d'exécution sont donc à la fois des outils de travail et des moyens de contrôle.

25. Tests de validation

Les tests doivent vérifier l'affichage simple, les fichiers cachés, le format long, les tris, les chemins multiples et les erreurs. Les cas de validation doivent couvrir à la fois les usages normaux et les situations fautives.

Comparer `./mys -la` à `ls -la` permet de mesurer l'écart entre l'implémentation pédagogique et l'utilitaire réel.

Cette étape confirme que le projet remplit bien sa fonction de reproduction partielle et compréhensible de `ls`.

26. Travail collaboratif

Le projet peut être réparti entre plusieurs membres : certains se chargent du parsing, d'autres du répertoire, du tri, de l'affichage ou de la gestion d'erreurs.

Git et GitHub facilitent cette collaboration grâce aux branches, commits, fusions et résolutions de conflits.

Une intégration fréquente est nécessaire pour éviter les divergences entre fichiers .c et .h.

27. Répartition des modules par membre

Le tableau ci-dessous associe chaque membre de l'équipe à son module de travail principal. Cette répartition facilite l'organisation du projet, la coordination des contributions et l'intégration finale du code :

MEMBRES	MODULES DE TRAVAIL
BITOUKOA Ousmane	main.c, options.c, options.h, intégration générale
AROUNG Ralph	directory.c, directory.h, entry.c, entry.h
MUKETE Mildred	display.c, display.h, utils.c, utils.h
NOUMBO Donang	sort.c, sort.h
JEAN-JOREST	error.c, error.h, tests, validation

28. Difficultés rencontrées

Les difficultés typiques concernent la mémoire dynamique, le parsing des options, l'utilisation de `Istat()`, le formatage de l'affichage long et le tri selon plusieurs critères.

Les problèmes d'intégration apparaissent aussi lorsque plusieurs membres modifient les mêmes prototypes ou les mêmes structures.

La compilation fréquente et la modularisation réduisent ces difficultés.

29. Solutions adoptées

Les solutions reposent sur la séparation nette des responsabilités, la vérification systématique des retours de fonctions, la centralisation des erreurs et les tests comparatifs réguliers.

Cette méthode correspond à une démarche d'ingénierie logicielle sérieuse, adaptée à un projet système.

Elle renforce à la fois la robustesse du code et la compréhension des mécanismes Linux.

30. Limites actuelles

Toutes les options de GNU ls ne sont pas forcément implémentées. Certaines fonctions avancées comme -R, -i, -s ou des formats très précis peuvent rester partiels ou simplifiés.

L'alignement exact du format long, les couleurs, les liens symboliques et certaines variantes de tri peuvent encore être améliorés.

Ces limites doivent être reconnues de manière transparente dans le rapport.

31. Perspectives d'amélioration

Le projet peut évoluer vers une meilleure conformité avec GNU ls, un support plus complet de la récursivité, une gestion plus avancée des couleurs et une amélioration du traitement des liens symboliques.

On peut également renforcer les tests automatisés et améliorer la documentation utilisateur.

Enfin, l'usage de fonctions standard comme `qsort()` peut être envisagé pour simplifier ou améliorer le tri.

Conclusion

Le projet myls montre clairement qu'une commande comme ls repose sur des mécanismes système réels : processus, appels POSIX, système de fichiers, permissions, mémoire et erreurs.

Rédigé sous la forme d'un cahier des charges techniques, le rapport met en évidence la portée pédagogique du projet et son lien direct avec le cours de Théorie des Systèmes d'Exploitation.